

Scalable E-commerce Platform Production Architecture, Scaling

Features, and Operating Model Date: 2025-08-10 UTC

Scalable E-commerce Platform Production Architecture, Scaling

Features, and Operating Model Date: 2025-08-10 UTC

One-Page Summary Goal Design a global, fault-tolerant e-commerce platform that delivers fast browsing, accurate availability and pricing, resilient checkout, and rich analytics—scaling from startup traffic to peak retail events while meeting strict security and privacy standards.

Key Outcomes

Sub-second page and API responses for core flows (browse, search, add-to-cart, checkout). Correct, low-latency inventory and pricing with safe reservations and idempotent payments. Multi-region cells with graceful degradation and zero data loss for accepted orders. Enterprise-grade security, privacy, and governance (SOC 2, GDPR, CCPA).

Primary Assumptions

Traffic ranges from tens of thousands to millions of RPS at peak (events, holidays, drops). Catalog size in the millions of SKUs; variants, options, and localized content. Multi-currency, multi-locale, multi-brand, marketplace sellers, and multiple fulfillment nodes. Reads >> writes; checkout path is write-heavy but narrow; search and browse are read-hot.

Functional Requirements

Customer Browse and search catalog; categories, facets, relevance, personalization. Product details, media, reviews, Q&A, recommendations, bundles. Cart and saved lists; guest and signed-in carts; promo codes and loyalty. Checkout with address capture, tax, shipping options, payments, and 3DS/SCA. Order lifecycle: confirmation, fulfillment, shipment tracking, returns/RMAs, refunds/store credit. Accounts: registration, auth, MFA, preferences, addresses, wallets, GDPR controls. Operations Catalog authoring, pricing rules and promotions engine, inventory management, drops/launches. Seller onboarding and feeds, sandbox to live workflows, SLAs and penalties. Customer support tooling, case management, appeasements. Data/Analytics Real-time KPIs and dashboards (conversion, AOV, margin, OOS rate), cohort analysis. Feeds and reverse ETL to marketing/CRM/ads platforms.

Non-

Availability p95: 99.95% for read APIs, 99.9% for writes, 99.99% for checkout redirect/confirm. Latency targets p95: search < 300 ms, PDP < 250 ms, add-to-cart < 120 ms, checkout step < 200 ms, order confirm < 500 ms. Durability: RPO 0 for accepted orders/payments; RTO ≤ 15 min per region. Scalability: horizontal across ingestion, search, caches, and order processing; cost-aware autoscaling. Compliance & privacy: SOC 2 Type 2, GDPR/CCPA; data minimization, retention, and residency.

Architecture Overview

Cell-Based, API-First Per-region “cells” host all runtime services, storage, and caches; a thin global control plane manages identity, configuration, routing, and observability. Clients and edges route to nearest healthy cell with anycast/geo DNS. Domain Microservices (CQRS where needed) Catalog, Media, Search, Pricing, Promotions, Inventory, Cart, Checkout, Orders, Payments, Fulfillment, Returns, Accounts/Auth, Reviews, Recs, CMS, Notifications, Fraud/Risk. Event Backbone Kafka/Kinesis for domain events (catalog updates, price changes, inventory deltas, order state). Debezium for CDC from relational stores. Edge & Web CDN, edge functions for A/B, bot mitigation, cache keys (locale/device), signed media URLs, image transforms. Orchestration Kubernetes, service mesh with mTLS and workload identity; API Gateway for authN/Z and rate limits.

Data Flow (Happy Path) 1) Browse/Search: Client → CDN → API Gateway → Search (OpenSearch) for candidates → hydrate from Catalog/Media → edge/cache. 2) Add to Cart: Client → Cart API (Redis write-through) → Inventory soft-hold token. 3) Checkout: Shipping/tax quote → Fraud scoring →

Payment auth (idempotency keys) → Order service persists, emits events → Fulfillment allocation → Confirmation. 4) Post-Order: Events feed analytics, notifications, and customer support systems.

Technology Stack (reference) Edge & Gateway: Cloud CDN, WAF/Bot manager, API Gateway (OAuth 2.1/OIDC, mTLS, DPOP). Compute: Kubernetes, autoscaling, HPA/KEDA on tail latency/lag; service mesh (SPIFFE/SPIRE). Search: OpenSearch/Elasticsearch for text/facets; RedisBloom for suggestions. Hot Caches: Redis Cluster for session/cart, derived views, rate limits. OLTP: Postgres/Aurora or Spanner for orders/accounting; DynamoDB/Scylla for carts/inventory state. Analytics: ClickHouse/BigQuery/Snowflake on S3/GCS with Iceberg/Delta; materialized views. Streaming: Kafka with Schema Registry; Flink/Spark Streaming for enrichment and fan-out. Payments: PCI-compliant PSPs via tokenization; webhooks with signature verification. Observability: OpenTelemetry, Prometheus/Grafana, Loki, Jaeger; SIEM for security signals. CI/CD & IaC: GitHub/GitLab, Argo CD/Rollouts, Terraform/Pulumi, Helm; SAST/DAST, SBOM, cosign.

Storage Strategy

Polyglot, CQRS-friendly Search index for discovery; Redis for hot paths (cart, sessions, derived reads); relational for correctness (orders, accounting, promotions); NoSQL/KV for high-QPS (carts, inventory state); object storage for media and analytics facts. Inventory & Orders Inventory uses a KV with conditional updates for soft-hold tokens (SET NX PX=ttl). Orders use relational with outbox pattern to publish events atomically. Payment and order states append-only for auditability. Analytics Lakehouse Bronze (raw events), Silver (normalized), Gold (curated marts). Compaction, clustering, time-travel; reverse ETL for growth tools.

Scaling Approach Sharding Keys tenant_id, region, and SKU/Cart hash to avoid hotspots. Separate read/write paths, denormalized read models (materialized projections) for PDP/cart/checkout. Autoscaling Signals Tail latency, queue lag, error budget burn. Pre-warm caches and search shards for scheduled drops. Backpressure & Degradation Serve cached PDP, reduce personalization, fall back to price/inventory snapshots, shrink search facets; queue non-essential writes (clicks) when under pressure. Latency Budgets & SLAs Edge/TLS/CDN < 30 ms p95 local. Search: 150–300 ms p95 (faceted). PDP hydrate: 120–250 ms p95. Add-to-cart: < 120 ms p95 (Redis + KV). Checkout step: < 200 ms p95; order confirm < 500 ms p95. Events E2E to consumers: < 3 s p95. Availability: Read 99.95%, Write 99.9%, Checkout path 99.99% for redirect/confirm endpoints.

Key Algorithms & Workflows Search/Relevance BM25 + popularity boosts; typo tolerance; synonym maps; category/facet pruning; dynamic reranking using click-through features. Pricing Priority evaluation of promotions; stackability, exclusion rules; currency normalization; price snapshots bound to cart line items. Inventory Reservation Soft-hold token on add-to-cart; confirm at payment authorization; periodic reconciliation with WMS events and snapshots. Fraud/Risk Velocity checks, device/IP reputation, ML-based scoring; step-up auth/SCA as needed. Payments Idempotency Idempotency keys and outbox for exactly-once effective capture/refund. Recommendations Stream features + near-real-time candidates; fallback to popular/related when cold.

Security, Reliability, Compliance Security TLS 1.3, HSTS, CSP, signed cookies, CSRF tokens; OAuth 2.1/OIDC; mTLS in mesh; secrets in KMS/Vault; WAF + bot protection; signed webhooks; input validation; SSRF egress controls. Privacy & Compliance PII minimization; per-tenant KMS keys; consent/DSAR APIs; residency routing; audit trails. Reliability Multi-AZ per cell; active-active stateless tiers; warm-standby for stateful stores; chaos and DR drills; error budgets and SLOs.

Multi-Tenancy Tenant id in every key; quotas/rate limits per tenant; per-tenant price lists, catalogs, and feature flags. Optional hard isolation: dedicated cells/VPCs. Per-tenant encryption keys and dashboards. CI/CD & Release Safety Everything as code; SAST/DAST/IAST; SBOM + signed images; canary rollouts gated on p95 latency, error rate, Kafka lag; automatic rollback on SLO burn. Online DDL with shadow writes/dual reads.

APIs & Integration Points Public Catalog browse/search, PDP, cart CRUD, checkout (shipping/tax/quote/submit), orders (status/returns), auth/account, promotions, recommendations, reviews, inventory availability, payments. Partner Seller feeds (catalog/pricing/inventory), WMS/OMS webhooks, PSP webhooks, tax/shipping rating. Admin Catalog authoring, rule editors, experiments, content

management, drops scheduling, analytics exports.

Observability & Operations OpenTelemetry traces/metrics/logs; golden dashboards: p95/p99 per domain, cache hit ratio, search shard health, cart latency, payment error rate, order backlog, inventory skew, event lag. SLO-aware autoscaling; budgets/cost guards. Runbooks: cache cold start, search shard failover, outbox replay, PSP incident, regional failover.

Capacity Planning & Cost Estimate (illustrative per busy region) Traffic: browse/search 50–150k RPS, checkout 2–10k RPS, events 100–300k EPS. Kafka: 6–9 brokers RF=3; 100–300 partitions for hot topics. Redis: 12–24 shards; keep < 65% memory; key TTL policies. Search: 6–12 data nodes; shard/replica tuned to index size and QPS. OLTP: 1 writer + 3–5 replicas; read/write splitting. Estimated monthly infra: 40k–150k depending on search size, retention, and media egress.

Testing Strategy & Failure Drills Unit/property tests for pricing, promotions, tax, and inventory math. Contract tests for public APIs and partner webhooks; schema registry gates. Load/soak for search, PDP hydrate, cart, and checkout; chaos for broker loss, Redis shard failover, cold cache, PSP outages. DR drills: restore orders/events, region evacuation; synthetic orders to validate end-to-end SLOs.

Data Models

(abridged) Product product_id, tenant_id, sku, variant_id, title, rich attributes JSON, media, category_ids, status, created_at, updated_at. Price product_id, tenant_id, list_price, sale_price, currency, region, channel, effective_from/to, price_list_id. Inventory tenant_id, warehouse_id, sku, on_hand, reserved, available, safety_stock, updated_at. Cart cart_id, tenant_id, user_id/null, items[{sku, qty, price_snapshot, promo_refs}], currency, ttl, version. Order order_id, tenant_id, user_id, items[{sku, qty, price, tax, discount}], totals, currency, payment_id, status, timeline[], addresses, shipment_groups[]. Payment payment_id, method_token, PSP refs, auth/capture/refund amounts, status, 3DS data, events[]. Promotion rule_id, scope, predicates, rewards, stacks_with, schedule, priority, audit. User user_id pseudonymous, auth refs, addresses, preferences, loyalty, consent. Event event_id, domain, key, payload_summary, created_at, version, producer, trace_id.

Phased Build Plan

MVP 3 months Single region cell; catalog/search, PDP, cart, checkout with one PSP; basic promotions; order service with outbox; analytics bronze; observability baseline. Phase 2 6–9 months Multi-region cells; inventory soft holds; richer promotions; returns/RMAs; seller feeds; analytics gold marts; canary delivery; fraud scoring. Phase 3 12 months Hard isolation options; advanced recommendations; global drops playbooks; cost optimizations; privacy vault; tighter SLOs and higher availability targets.

Key Choices, Trade-Offs, Risks, Mitigations Search vs cost: dedicated OpenSearch gives speed but costs; mitigate with cache/mv tables and shard hygiene. Inventory accuracy vs latency: soft hold tokens keep UX fast; reconcile to guarantee correctness. Event backbone adds ops cost; payoff is replayability and loose coupling. Third-party PSP/webhook variability: isolate via queues/outbox and retries; surface circuit breakers. Personalization vs privacy: use on-device/edge features when possible; anonymize/aggregate server-side.